

Does Optimization Parallel Inference Change LLM Inference Outputs?

A Concise Empirical Report on vLLM Configuration Effects

CSE 5429

May 8, 2026

TL;DR

We test whether vLLM system-level inference optimizations change LLM outputs when the model, prompts, seed, and greedy decoding setup are fixed. Among the nine configurations that completed, six match the baseline exactly on all 20 prompts. Two configurations change outputs: CUDA graphs match only 25% of baseline outputs, and batch processing matches 50%. The baseline determinism check is perfectly repeatable, so the observed differences are tied to execution configuration rather than ordinary run-to-run randomness.

1 Question and Motivation

Modern LLM serving systems improve throughput and latency with optimizations such as chunked prefill, CUDA graphs, prefix caching, batching, and CPU offloading. These optimizations are usually treated as implementation details: they should make inference faster, not change the generated text. This report tests that assumption directly.

The core question is simple: *given the same model and prompt, do different inference configurations produce the same output?* This matters because output reproducibility affects evaluation, debugging, and safety analysis. If a model answer changes when the serving path changes, then inference configuration is part of the experimental condition, not merely an engineering detail.

2 Experimental Setup

The experiment uses `Qwen/Qwen2-1.5B-Instruct` served with vLLM v0.20.1 on a single NVIDIA RTX 4090. All completed runs use greedy decoding with temperature = 0, seed = 42, maximum context length 2048, and maximum generation length 128. The prompt set contains 20 prompts spanning short factual, medium creative, long reasoning, code-generation, edge-case, and repetition-sensitive inputs. Each inference configuration is run in an isolated subprocess to avoid GPU-memory state carrying over between configurations.

Table 1 summarizes the tested configurations. C7 is optional and was skipped in the final run because FP8 KV cache initialization failed in this environment.

Outputs are compared with exact match, token match ratio, BLEU, ROUGE-L, edit distance, and first divergent token position. Exact match is the main metric: it directly answers whether two generations are character-for-character identical.

Table 1: Inference configurations.

ID	Name	Purpose
C0	Baseline	Reference configuration with eager execution
C1	Chunked Prefill (512)	Tests small prefill chunking
C2	Chunked Prefill (2048)	Tests larger prefill chunking
C3	CUDA Graphs	Tests CUDA graph execution rather than eager mode
C4	Prefix Caching	Tests KV cache prefix reuse
C5	Batch Processing	Tests batched prompt scheduling
C6	Low GPU Memory	Tests lower GPU memory utilization (40%)
C7	FP8 KV Cache	Optional cache-precision test; skipped
C8	CPU Offload	Tests 4 GB CPU offload
C9	Determinism	Repeats baseline three times

3 Results

Nine of ten configurations completed. C7 was skipped after FP8 KV cache initialization failed; all other configurations produced 20 outputs, except C9, which produced 60 outputs because it repeats the baseline three times. Table 2 reports each completed non-baseline configuration against C0.

Table 2: Comparison against C0 baseline, averaged over 20 prompts.

Config	Exact Match	BLEU	Edit Dist.	First Div.
C1 Chunked (512)	1.000	100.0	0.0	—
C2 Chunked (2048)	1.000	100.0	0.0	—
C3 CUDA Graphs	0.250	69.7	154.2	28.0
C4 Prefix Cache	1.000	100.0	0.0	—
C5 Batch Proc.	0.500	78.7	111.5	16.7
C6 Low Memory	1.000	100.0	0.0	—
C8 CPU Offload	1.000	100.0	0.0	—
C9 Determinism	1.000	100.0	0.0	—

The main result is sharp: chunked prefill, prefix caching, low memory, and CPU offload are identical to the baseline; CUDA graphs and batch processing are not. CUDA graphs match only 5 of 20 prompts exactly. Batch processing matches 10 of 20 prompts exactly. C9 confirms that the baseline is stable across repeated runs, so the C3 and C5 differences are not explained by ordinary nondeterministic reruns of C0.

4 Discussion

The results suggest a useful distinction between *memory-management* optimizations and *execution-path* optimizations. The memory-oriented changes tested here—prefix caching, lower GPU memory utilization, and CPU offload—do not change outputs. They appear transparent for this model, prompt set, and decoding setup.

CUDA graphs and batch processing are different. Both change how GPU work is scheduled or replayed. A plausible explanation is numerical sensitivity: different kernel execution paths can introduce small floating-point differences, and greedy decoding can amplify a small logit-order change into a different token choice. This explanation is consistent with the observations, especially

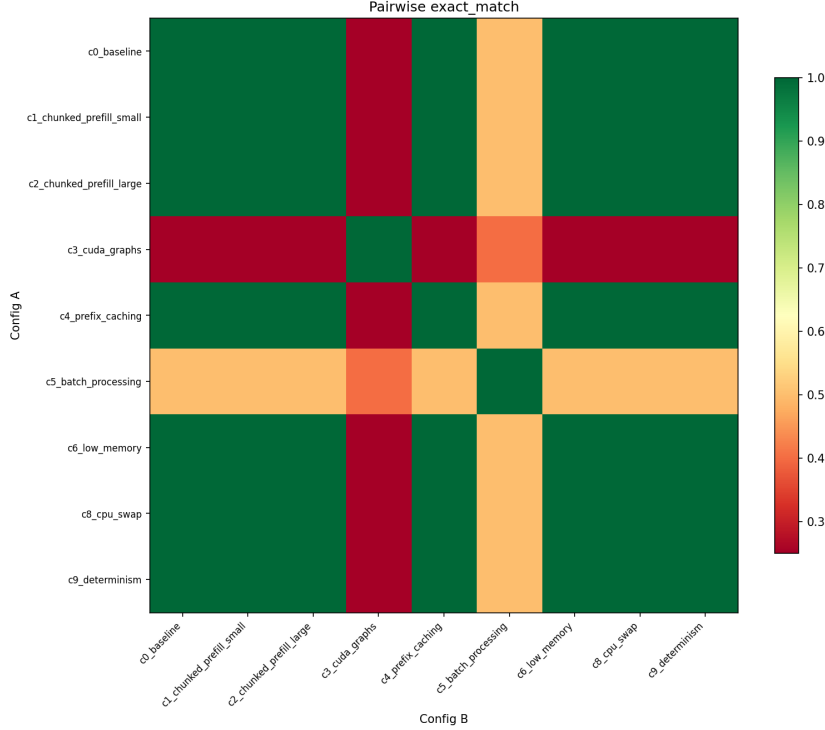


Figure 1: Pairwise exact match heatmap across completed configurations.

the nonzero first-divergence positions, but it is not directly proven by these experiments because we do not log intermediate logits or kernel-level numeric differences.

The practical implication is that reproducibility-sensitive evaluations should record the inference configuration, not only the model and prompt. In this experiment, the same model and greedy decoding setup can produce different text when CUDA graphs or batching are enabled. For throughput-oriented production systems this may be acceptable; for exact reproducibility it is a real experimental variable.

5 Limitations

This is a focused single-system experiment. The conclusions are based on one model, one GPU, one vLLM version, greedy decoding, and 20 prompts. The FP8 KV cache condition did not complete in this environment. Larger models, multi-GPU serving, sampling-based decoding, and direct logit comparison would be useful follow-up experiments.

6 Conclusion

Most tested vLLM optimizations preserve exact outputs under this setup, but not all do. CUDA graphs and batch processing changed generated text even with a fixed seed and temperature = 0, while the baseline itself was perfectly repeatable. The central insight is therefore not that LLM inference is always unstable, but that exact output reproducibility depends on the execution path.